title: "Day 1 — TypeScript Fundamentals" subtitle: "Variables · Data Types · Naming Conventions · Interview Q&A" author: "TypeScript + Playwright Learning Series" date: "October 2026"

📗 DAY 1 — TypeScript Fundamentals

TypeScript + Playwright Learning Series

Chapter	Topic
Chapter 1	Variables in TypeScript
Chapter 2	Data Types in TypeScript
Chapter 3	Naming Conventions
Chapter 4	Interview Questions & Answers

> **IDE Shortcuts Learned on Day 1:**
>
> - `F2` → Rename a file
> - `Ctrl + L` → Open AI Agent
> - Type `log` + `Tab/Enter` → Auto-complete `console.log()`

Chapter 1: Variables in TypeScript

What is a Variable?

A variable is a **named container** that stores a value in memory so you can use and reuse it in your program.

Format / Syntax:

```
keyword variableName: type = value;
```

1. Declaration Keywords

TypeScript (and modern JavaScript) provides **3 keywords** to declare variables:

Keyword	Reassignable?	Scope	Recommendation
`let`	✅ Yes	Block-scoped	✅ Use when value will change
`const`	❌ No	Block-scoped	✅ **Default choice** — value stays fixed
`var`	✅ Yes	Function-scoped	⛔ Avoid — outdated, causes bugs

2. `let` — Reassignable Variable

Use `let` when the value **needs to change** over time.

```
let studentName = 'Vikas';
console.log(studentName); // Vikas

studentName = 'Ravi'; // ✅ Allowed — value can be updated
console.log(studentName); // Ravi
```

3. `const` — Constant (Cannot be Reassigned)

Use `const` when the value **should never change** after being set.

```
const country = 'India';
console.log(country); // India

country = 'Arab'; // ❌ Error: Cannot assign to 'country' because it is a constant.
```

> 💡 **Best Practice:** Always start with `const`. Only switch to `let` if you actually need to change the value later.

---

4. `var` — Don't Use It!

`var` is the old way of declaring variables in JavaScript. It has quirky behavior (function scope + hoisting) that causes hard-to-find bugs.

```
var oldStyle = 'Avoid this';   // ⛔ Works, but don't use it
```

Why avoid `var`?

- It is **function-scoped**, not block-scoped (leaks out of `if` / `for` blocks)
- It gets **hoisted** (moved to the top of its scope silently)
- `let` and `const` solve both problems

---

5. Type Annotations (Optional but Powerful)

TypeScript lets you explicitly declare what **type** a variable holds.

```
keyword   variableName : type     = value;
let       studentName  : string   = 'Vikas';
```

Primitive Types

Type	Example	Description
`string`	`'Vikas'`, `"India"`	Text
`number`	`30000`, `1.45`	Integer or decimal
`boolean`	`true`, `false`	True or false only
`null`	`null`	Intentionally empty
`undefined`	`undefined`	Not yet assigned

```
let testEnvironment: string  = 'Staging';
let timeout:         number  = 30000;
let isHeadless:      boolean = true;
```

---

6. Type Inference (TypeScript Guesses the Type)

If you assign a value when declaring, TypeScript **automatically infers** the type — you don't need to write it explicitly.

```
let studentName = 'Vikas';  // TypeScript knows this is a string
let retryCount  = 3;        // TypeScript knows this is a number

studentName = 100; // ❌ Error: Type 'number' is not assignable to type 'string'
```

---

7. Template Literals & `console.log()`

Use `console.log()` to display variable values in the terminal.

```
let studentName = 'Vikas';
const country   = 'India';

console.log(studentName);          // Vikas
console.log(country);              // India
console.log('Name:', studentName); // Name: Vikas

// Template literal — embed variables inside a string using backticks
console.log(`Student ${studentName} is from ${country}`);
// Output: Student Vikas is from India
```

> **Shortcut:** Type `log` and press `Tab` or `Enter` → it auto-expands to `console.log()`

Chapter 1 Summary

Concept	Key Point
`const`	Value **cannot** change — use by default
`let`	Value **can** change — use when needed
`var`	**Avoid** — outdated, block-scope bugs
Type annotation	`let name: string = 'Vikas'`
Type inference	TypeScript auto-detects type from value
`console.log()`	Prints output to the terminal

Chapter 2: Data Types in TypeScript

TypeScript is a **strongly typed** language — every variable has a type, and TypeScript enforces it at compile time (before your code runs). This catches bugs early and makes your code much safer.

Overview: TypeScript Data Types

```
TypeScript Data Types
│
├── Primitive Types (stored by VALUE)
│   ├── number
│   ├── string
│   ├── boolean
│   ├── null
│   ├── undefined
│   ├── bigint
│   └── symbol
│
└── Non-Primitive / Reference Types (stored by REFERENCE)
    ├── object
    ├── array
    ├── tuple
    ├── enum
    ├── any / unknown
    └── never / void
```

1. `number` — All Numeric Values

TypeScript has **one** number type for both integers and decimals (unlike Java which has `int`, `float`, `double`).

```
let a: number = 10;          // integer
let price: number = 99.99;   // decimal / float
let negative: number = -5;   // negative number
```

> 💡 Use `number` for timeouts, counts, prices, ages — anything numeric.

2. `string` — Text Data

A string holds any **text** value. You can use single quotes, double quotes, or backticks.

```
let studentName: string = 'Harsh Zavery';        // single quotes
let city: string = "Mumbai";                      // double quotes
let greeting: string = `Hello, ${studentName}!`; // template literal
```

Quote Style	Syntax	Use For
Single `'`	`'text'`	Regular strings
Double `"`	`"text"`	Regular strings
Backtick `` ` ``	`` `Hello ${name}` ``	Template literals with variables

3. `boolean` — True or False

A boolean can only be **one of two values**: `true` or `false`.

```
let isMarried: boolean = true;
let isHeadless: boolean = false;

if (isMarried) {
    console.log('Married');
} else {
    console.log('Single');
}
```

> 💡 In automation: `isHeadless`, `isPassed`, `isLoggedIn`, `isVisible` are all booleans.

4. `null` — Intentionally Empty

`null` means a variable has been **deliberately set to have no value**.

```
let cart: null = null;

console.log(cart);        // null
console.log(typeof cart); // "object"  ← a famous JavaScript quirk!
```

> ⚠️ `typeof null` returns `"object"` — this is a well-known JavaScript bug.

---

5. `undefined` — Not Yet Assigned

`undefined` means a variable has been **declared but not yet given a value**.

```
let product: undefined = undefined;
console.log(product); // undefined
```

`null` vs `undefined` — Key Differences

	`null`	`undefined`
Meaning	Intentionally empty	Not yet given a value
Set by	Developer (on purpose)	JavaScript engine (by default)
`typeof`	`"object"` ⚠️	`"undefined"`

```
console.log(null == undefined);  // true  (loose equality)
console.log(null === undefined); // false (strict equality — different types!)
```

---

6. `bigint` — Very Large Integers

`bigint` is used for numbers **larger than** `Number.MAX_SAFE_INTEGER`. Add `n` at the end.

```
let b: bigint = 10000000n;

console.log(b); // 10000000n
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

> ⚠️ You **cannot** mix `bigint` and `number`:
>
> ```
> let result = b + 10;  // ❌ Error
> let result = b + 10n; // ✅ Correct
> ```

---

7. `symbol` — Unique Identifiers

A `symbol` creates a **guaranteed unique value**, even if two symbols have the same description.

```
let id: symbol = Symbol('id');
let id2: symbol = Symbol('id');

console.log(id === id2); // false — always unique!
```

All Primitives at a Glance

```
let a: number    = 10;
let studentName: string  = 'Harsh Zavery';
let isMarried: boolean = true;
let cart: null     = null;
let product: undefined = undefined;
let b: bigint    = 10000000n;
let id: symbol   = Symbol('id');
```

Type Checking at Runtime — `typeof`

Type	`typeof` result
`number`	`"number"`
`string`	`"string"`
`boolean`	`"boolean"`
`null`	`"object"` ⚠️
`undefined`	`"undefined"`
`bigint`	`"bigint"`
`symbol`	`"symbol"`

Chapter 2 Summary

- TypeScript's type system catches errors **before** your code runs.
- **7 primitive types**: `number`, `string`, `boolean`, `null`, `undefined`, `bigint`, `symbol`.
- `null` = intentionally empty | `undefined` = not yet assigned.

- `bigint` is for numbers too large for the regular `number` type — add `n` suffix.
- `typeof null` returns `"object"` — famous JavaScript bug, not a TypeScript mistake.

Chapter 3: Naming Conventions in TypeScript

Naming conventions are **agreed-upon rules** for how to name variables, functions, classes, files, etc. Following them makes your code **readable**, **consistent**, and **professional**.

The 4 Main Naming Styles

Style	Pattern	Example
camelCase	First word lowercase, rest capitalized	`studentName`, `isMarried`
PascalCase	Every word starts with capital	`TestConfig`, `LoginPage`
UPPER_SNAKE_CASE	All caps, words joined by `_`	`MAX_TIMEOUT`, `BASE_URL`
kebab-case	All lowercase, words joined by `-`	`login-page.ts`

1. Variables & Parameters → `camelCase`

```
// ✅ Correct
let studentName: string = 'Harsh Zavery';
let isMarried: boolean = true;
let timeoutMs: number = 30000;

// ❌ Wrong
let StudentName = 'Harsh';   // PascalCase — reserved for classes
let student_name = 'Harsh';  // snake_case — not TypeScript style
```

Boolean Variables — Use a Prefix

```
let isHeadless: boolean = true;       // ✅ is ...
let hasPermission: boolean = false;   // ✅ has ...
let canRetry: boolean = true;         // ✅ can ...
let shouldSkip: boolean = false;      // ✅ should ...
```

2. Constants → `UPPER_SNAKE_CASE`

```
const MAX_TIMEOUT: number = 30000;
const BASE_URL: string = 'https://demo.playwright.dev';
const MAX_RETRY_COUNT: number = 3;
```

> 💡 Use for configuration values shared across files.

3. Functions → `camelCase` (Verb First)

```
// ✅ Correct — verb + noun
function loginUser() { }
function getStudentName() { }
function isUserLoggedIn() { }  // boolean-returning function

// ❌ Wrong
function LoginUser() { }       // PascalCase (looks like a class)
function user() { }            // no verb — unclear purpose
```

4. Classes & Interfaces → `PascalCase`

```
class StudentProfile { }
class BrowserManager { }

interface TestConfig {
    baseUrl: string;
    timeout: number;
    isHeadless: boolean;
}

type BrowserType = 'chromium' | 'firefox' | 'webkit';
```

5. Enums → `PascalCase`

```
enum Browser {
    Chromium = 'chromium',
    Firefox  = 'firefox',
    Webkit   = 'webkit',
}

enum TestStatus {
    Passed  = 'PASSED',
    Failed  = 'FAILED',
    Skipped = 'SKIPPED',
}
```

6. Files & Folders → `kebab-case`

```
✅ Correct:
src/Day1/variables.ts
src/Day1/data-types.ts
src/page-objects/login-page.ts

❌ Wrong:
src/Day1/Variables.ts          ← PascalCase
src/Day1/dataTypes.ts          ← camelCase
src/Day1/naming_conventions.ts ← snake_case
```

7. No `I` Prefix on Interfaces

```
// ⛔ Old Java-style — avoid
interface ITestConfig { }

// ✅ Modern TypeScript style
interface TestConfig { }
```

Full Naming Reference Table

Thing	Convention	Example
Variable	`camelCase`	`studentName`, `timeoutMs`
Boolean	`camelCase` + prefix	`isHeadless`, `hasPermission`
Function	`camelCase` + verb	`loginUser()`, `getTimeout()`
Class	`PascalCase`	`LoginPage`, `BrowserManager`
Interface	`PascalCase`	`TestConfig`, `UserCredentials`
Type alias	`PascalCase`	`BrowserType`, `TestStatus`
Enum	`PascalCase`	`enum Browser { Chromium }`
Constant	`UPPER_SNAKE_CASE`	`MAX_TIMEOUT`, `BASE_URL`
File	`kebab-case`	`login-page.ts`
Folder	`kebab-case`	`page-objects/`

Quick Rules to Remember

> ✅ Variables & functions → `camelCase` ✅ Classes, Interfaces, Types, Enums → `PascalCase` ✅ Config constants → `UPPER_SNAKE_CASE` ✅ Files & folders → `kebab-case` ✅ Booleans → prefix with `is`, `has`, `can`, `should` ⛔ No `I` prefix on interfaces ⛔ No `snake_case` for variables

Chapter 4: Interview Questions & Answers

> **Difficulty levels:** 🟢 Basic · 🟡 Intermediate · 🔴 Tricky / Gotcha

🟢 Variables — Basic

Q1. What is a variable in TypeScript?

A variable is a **named container** that stores a value in memory. TypeScript enforces a **type** on every variable, catching bugs at compile time.

```
let studentName: string = 'Vikas';
```

Q2. What are the three keywords to declare variables?

`let`, `const`, and `var`.

Keyword	Reassignable	Scope	Use
`let`	✅ Yes	Block	Value will change
`const`	❌ No	Block	Value stays fixed
`var`	✅ Yes	Function	⛔ Avoid

Q3. What is the difference between `let` and `const`?

- `let` → value **can be changed** after declaration.
- `const` → value **cannot be reassigned** after being set.

```
let city = 'Mumbai';
city = 'Delhi'; // ✅ OK

const country = 'India';
country = 'USA'; // ❌ Error: Cannot assign to 'country'
```

Q4. Why should we avoid `var`?

1. **Function scope** — leaks out of `if` / `for` blocks.
2. **Hoisting** — silently moved to the top of its function.

```
if (true) {
    var x = 10; // leaks!
    let y = 20; // stays inside
}
console.log(x); // 10  ← leaked
console.log(y); // ❌ ReferenceError
```

🟡 Variables — Intermediate

Q5. What is Type Inference?

TypeScript **automatically determines the type** from the assigned value.

```
let studentName = 'Vikas'; // inferred as string
let retryCount  = 3;       // inferred as number

studentName = 100; // ❌ Error: number not assignable to string
```

Q6. Can you change the properties of a `const` object?

Yes! `const` only prevents **reassigning the variable** — not mutating its contents.

```
const config = { browser: 'chromium', headless: true };

config.headless = false;     // ✅ OK — mutating a property
config = { browser: 'ff' };  // ❌ Error — reassignment blocked
```

Q7. What is a Union Type?

A Union Type lets a variable hold **more than one type** using `|`.

```
let timeout: number | string = 5000;
timeout = '10s'; // also valid!

let status: 'Passed' | 'Failed' | 'Skipped' = 'Passed';
```

🟢 Data Types — Basic

Q8. What are the 7 primitive types in TypeScript?

`number`, `string`, `boolean`, `null`, `undefined`, `bigint`, `symbol`

```
let a: number    = 10;
let name: string = 'Harsh';
let ok: boolean  = true;
let x: null      = null;
let y: undefined = undefined;
let b: bigint    = 10000000n;
let id: symbol   = Symbol('id');
```

Q9. What is the difference between `null` and `undefined`?

	`null`	`undefined`
Meaning	Intentionally empty	Not yet assigned
Set by	Developer	JS engine
`typeof`	`"object"` ⚠️	`"undefined"`

```
console.log(null == undefined);  // true  (loose)
console.log(null === undefined); // false (strict)
```

Q10. What is `bigint` and how do you create one?

`bigint` holds integers larger than `Number.MAX_SAFE_INTEGER`. Suffix with `n`.

```
let b: bigint = 10000000n;

// Cannot mix with number:
let r = b + 10;  // ❌ Error
let r = b + 10n; // ✅ OK
```

🟡 Data Types — Intermediate

Q11. What is the difference between `any` and `unknown`?

	`any`	`unknown`
Type safety	❌ None	✅ Enforced
Operations	Anything, no checks	Must check type first

```
let x: any = 'hello';
x = 100;
x.toUpperCase(); // ❌ Runtime crash — TypeScript didn't warn!

let y: unknown = 'hello';
if (typeof y === 'string') {
    y.toUpperCase(); // ✅ Safe
}
```

Q12. What does `typeof` return for each type?

```
typeof 10          // "number"
typeof 'hello'     // "string"
typeof true        // "boolean"
typeof null        // "object"  ⚠️ Bug!
typeof undefined   // "undefined"
typeof 10n         // "bigint"
typeof Symbol()    // "symbol"
```

🟢 Naming Conventions

Q13. What convention do variables use? `camelCase` — e.g., `studentName`, `isHeadless`, `maxRetry`

Q14. What convention do classes and interfaces use? `PascalCase` — e.g., `LoginPage`, `TestConfig`, `BrowserType`

Q15. When should you use `UPPER_SNAKE_CASE`? For configuration constants shared across files — e.g., `MAX_TIMEOUT`, `BASE_URL`

Q16. How should boolean variables be named? With a prefix: `isHeadless`, `hasPermission`, `canRetry`, `shouldSkip`

🔴 Tricky / Gotcha Questions

Q17. 🔴 What does `typeof null` return, and why?

Returns `"object"` — a **famous JavaScript bug** from 1995. The internal bit pattern for `null` matched object references. Never fixed to avoid breaking existing sites.

```
// ❌ Wrong way to check for null
if (typeof value === 'object') { } // matches real objects too!

// ✅ Correct way
if (value === null) { }
```

Q18. 🔴 `const` means a variable can't change — True or False?

Partially True. `const` prevents **reassignment** but NOT **mutation** of object contents.

```
const user = { name: 'Vikas' };
user.name = 'Harsh';    // ✅ Mutation — OK!
user = { name: 'New' }; // ❌ Reassignment — Error!
```

Q19. 🔴 Does TypeScript remove types at runtime?

Yes. TypeScript types exist **only at compile time**. The compiled JavaScript has no type annotations.

```
// TypeScript
let name: string = 'Vikas';

// Compiled JavaScript (type stripped)
let name = 'Vikas';
```

Q20. 🔴 Is `let` hoisted like `var`?

`let` IS hoisted, but falls in the **Temporal Dead Zone (TDZ)** — it exists but cannot be accessed before its declaration.

```
console.log(a); // undefined (var — hoisted & accessible)
var a = 10;

console.log(b); // ❌ ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

Q21. 🔴 Can two `Symbol('id')` values ever be equal?

No. Symbols are always unique regardless of description.

```
let id1 = Symbol('id');
let id2 = Symbol('id');
console.log(id1 === id2); // false — always!
```

Q22. 🔴 What happens when you declare `let x;` with no value or type?

TypeScript assigns it the type `any` implicitly. With `"strict": true`, TypeScript throws an error.

```
let x;        // type: any (implicit)
x = 'hello';  // fine
x = 42;       // fine — no type safety!

// strict mode:
let y;        // ❌ Error: Variable 'y' implicitly has an 'any' type
```

⚡ Quick Fire Round

Question	Answer
`typeof null` returns?	`"object"` ⚠️
TS types available at runtime?	❌ No — stripped at compile
`bigint` suffix?	`n` — e.g., `10n`
Can `const` object props change?	✅ Yes — only reassignment blocked
`null == undefined`?	✅ `true` (loose)
`null === undefined`?	❌ `false` (strict)
Can two `Symbol('id')` be equal?	❌ Never
Variables naming style?	`camelCase`
Class naming style?	`PascalCase`
Config constants naming style?	`UPPER_SNAKE_CASE`
File naming style?	`kebab-case`
`I` prefix on interfaces?	⛔ Avoid in modern TypeScript
`let` hoisted?	✅ Yes — but in TDZ until declared
Prefer `any` or `unknown`?	✅ `unknown` — safer
Default type of untyped variable?	`any` (implicit)

💡 Interview Tips

1. **Back up every answer with a code snippet** — interviewers love seeing code.
2. **Mention gotchas** — e.g., `const` with objects, `typeof null`.
3. **Compare opposites** — `null vs undefined`, `any vs unknown`, `let vs const`.
4. **Use automation context** — relate answers to Playwright/testing where possible.
5. **Be honest if unsure** — *"I believe it works like X, let me think through the edge case."*

📗 *End of Day 1 — TypeScript Fundamentals*

TypeScript + Playwright Learning Series